目标 — 框架 — 方法

欧拉定理的 Lean 形式化证明

对任意自然数 n，都存在严格大于 n 的素数

本文把“素数有无穷多个”写成 Lean 命题，并用 `n! + 1` 的最小素因子构造出严格大于 `n` 的素数，全程不使用 `sorry`。

01 目标 GOAL

命题：对每个自然数 `n`，存在素数 `p`，使得 `p > n`。这里不限定 `n` 的范围，也不额外假设 `p` 与 `n` 互素或满足其他条件；任务只有两个：构造出 `p`，并证明它确实是素数且严格大于 `n`。

在 Lean 中，“存在”不是一句口号：必须给出具体的项 `p`，再给出 `Nat.Prime p` 与 `n < p` 的证明项。下面这条 `theorem` 就是全篇要闭合的目标类型。

定理（目标类型）

```
theorem exists_prime_gt (n : ℕ) :
    ∃ p : ℕ, Nat.Prime p ∧ n < p
```

01 对任意 n	02 见证 p	03 两条性质
无额外前提；n = 0, 1 时 p = 2	由 n! + 1 构造出来	Nat.Prime p 与 n < p

02 框架 FRAMEWORK

核心思想是欧几里得的构造：把前 n 个自然数的乘积加一。任何整除这个数的素数，都不可能整除 `n!`，因此它只能大于 `n`。

Lean 中把“取一个素因子”实现为取最小素因子 `p = (n! + 1).minFac`，既具体又可计算。

① 构造 `N = n! + 1` 且 N > 1 → ② 取素因子 `p = N.minFac` p 是素数 → ③ 反设 `p ≤ n` 则 p ∣ n! → ④ 矛盾 `p ∣ 1` 故 n < p

为什么是 `n! + 1`？因为 `n!` 含有所有不超过 `n` 的正因子；加一后，任何不超过 `n` 的素数都不可能同时整除两者。于是 `n! + 1` 的最小素因子只能落在 `n` 的右侧。

A 形式化对象与引理分工

Lean 对象	类型 / 表达式	在证明中的作用
`N`	`n.factorial + 1`	大于 1 的构造数；N ≠ 1
`p`	`N.minFac`	N 的最小素因子，作为见证
`hN`	`N ≠ 1`	保证 minFac 真的是素数
`hpdvd`	`p ∣ N`	把 p 与构造数连接起来
`hpfac`	`p ∣ n!`	由反设 p ≤ n 与阶乘性质推出
`hp1`	`p ∣ 1`	与“素数不能整除 1”矛盾

形式化策略： 存在性是构造性的，`p` 直接由 `N.minFac` 给出；反证只用于最后的大小比较——假设 `p ≤ n`，就会同时得到 `p ∣ n!` 与 `p ∣ n! + 1`，从而逼出 `p ∣ 1`。

03 方法 METHOD

A 数学证明：五步闭合目标

① **构造。**令 `N = n! + 1`。因为 `n! ≥ 1`，所以 `N ≥ 2`，特别地 `N ≠ 1`。

② **取最小素因子。**令 `p = minFac(N)`。由最小素因子的基本性质，`p` 是素数，并且 `p ∣ N`。

③ **反设。**假设结论不成立，即 `p ≤ n`。这是全篇唯一一次反证。

④ **导出整除。**因为 `p > 0` 且 `p ≤ n`，阶乘定义给出 `p ∣ n!`；另一方面 `p ∣ N = n! + 1`。

⑤ **矛盾。**于是 `p` 整除 `(n! + 1) − n! = 1`。但素数不能整除 1，矛盾。故 `p > n`。

结论：见证 `p` 已经构造，`Nat.Prime p` 与 `n < p` 均已证明，目标类型闭合。

B Lean 4 代码：与五步一一对应

下面的脚本把上述论证逐行写成证明项。它沿用 Mathlib 中 `Nat.exists_infinite_primes` 的 Euclid 构造骨架，但没有调用该定理本身。

Euclid.lean Lean 4 · Mathlib

```
import Mathlib.Data.Nat.Prime.Infinite

namespace Euclid

/-- 对任意 n，存在严格大于 n 的素数。 -/
theorem exists_prime_gt (n : ℕ) :
    ∃ p : ℕ, Nat.Prime p ∧ n < p := by
  -- ① 构造 N = n! + 1
  let p := (n.factorial + 1).minFac
  have hN : n.factorial + 1 ≠ 1 :=
    ne_of_gt (Nat.succ_lt_succ (Nat.factorial_pos n))
  -- ② 取最小素因子
  have hp : Nat.Prime p := Nat.minFac_prime hN
  have hpdvd : p ∣ n.factorial + 1 := Nat.minFac_dvd _
  -- ③④⑤ 反设 p ≤ n，推出 p ∣ 1
  have hlt : n < p := by
    by_contra h
    have hle : p ≤ n := Nat.le_of_not_gt h
    have hpfac : p ∣ n.factorial :=
      Nat.dvd_factorial (Nat.minFac_pos _) hle
    have hp1 : p ∣ 1 := (Nat.dvd_add_iff_right hpfac).2 hpdvd
    exact hp.not_dvd_one hp1
  exact ⟨p, hp, hlt⟩

end Euclid
```

步骤对照：① → `hN`；② → `hp`、`hpdvd`；③ → `by_contra h` 与 `hle`；④ → `hpfac`；⑤ → `hp1`、`hp.not_dvd_one`。

目标闭合：`⟨p, hp, hlt⟩` 同时给出见证 `p`、素性证明与严格大于的证明。

> **复核与说明**
>
> 脚本中没有 `sorry` 或 `admit`。在已安装 Mathlib 的项目中保存为 `Euclid.lean`，运行 `lake env lean Euclid.lean`；无错误输出即表示证明通过 Lean 内核检查。
>
> 术语说明：题中“欧拉定理”指素数无穷多这一陈述。历史上它通常称欧几里得定理（Euclid's theorem）；欧拉另有基于调和级数发散的著名证明。

参考：Mathlib4 · Mathlib/Data/Nat/Prime/Infinite.lean